

Checklist:

Team Topologies – Next-Gen Operating Model

1. Diagnose Current Delivery Problems

- ☐ Teams are not disengaged or overloaded.
- ☐ Software delivery is not centered only on “feature delivery.”
- ☐ Team boundaries are clear.
- ☐ Reorganizations are neither frequent nor disruptive, nor poorly explained.
- ☐ Conway’s Law is understood and intentionally used.
- ☐ Architectural choices are not accidental or driven by friction.
- ☐ Teams are not isolated in ways that prevent effective scaling.
- ☐ Cognitive load is visible and actively managed.

2. Start with Team-First Thinking

- ☐ Treat the team as the fundamental delivery unit.
- ☐ Ensure each team has a clear purpose and mission.
- ☐ Give teams reasonable autonomy.
- ☐ Make sure teams effectively own their part of the software.
- ☐ Design work so teams can focus on user needs.
- ☐ Reduce unnecessary cognitive load.
- ☐ Ensure teams can consume and provide software or information to other teams.
- ☐ Include documentation, onboarding, support, and communication in the team’s responsibility.

3. Choose the Right Team Types

- ☐ Identify **stream-aligned teams** for the main flow of business change.
- ☐ Identify **platform teams** to support stream-aligned teams.
- ☐ Use **enabling teams** to help other teams learn or adopt new practices.
- ☐ Use **complicated-subsystem teams** only where specialist knowledge is truly needed.
- ☐ Avoid creating additional team types unless absolutely necessary.
- ☐ Ensure every team type has a clear reason to exist.

4. Define Team Interaction Modes

- ☐ Use **collaboration mode** when teams need to discover new approaches together.
- ☐ Use **X-as-a-Service mode** when one team consumes a service, API, tool, or product from another.
- ☐ Use **facilitating mode** when one team helps another adopt or learn something.
- ☐ Avoid unclear or ad hoc interaction patterns.
- ☐ Regularly review whether current interaction modes are still appropriate.

5. Apply Conway's Law Strategically

- ☐ Align team structure with the desired software architecture.
- ☐ Reduce unnecessary communication between teams.
- ☐ Design teams so systems can be deployed independently.
- ☐ Avoid large upfront architecture designs that ignore team communication patterns.
- ☐ Make ownership of code, services, and APIs clear.
- ☐ Use team boundaries to support modular, maintainable software.

6. Identify Streams of Change

- ☐ Identify the main business flows where change happens.
- ☐ Align stream-aligned teams to those flows.
- ☐ Choose streams based on real business needs, not internal departments.
- ☐ Consider possible stream types:
 - ☐ Customer or citizen tasks

- ☐ Business products
- ☐ Online purchasing journeys
- ☐ Regional markets
- ☐ Market segments
- ☐ Confirm that each stream reflects meaningful change pressure.

7. Build a Thinnest Viable Platform

- ☐ Identify the minimum platform services needed by stream-aligned teams.
- ☐ Keep the platform “just big enough.”
- ☐ Avoid building a large platform before teams need it.
- ☐ Provide clear documentation and onboarding.
- ☐ Encapsulate underlying complexity.
- ☐ Support reliable, fast flow of change.
- ☐ Allow the platform to evolve over time.

8. Manage Cognitive Load and Architecture Size

- ☐ Limit subsystem size to what one team can understand and own.
- ☐ Avoid architecture that overwhelms teams.
- ☐ Design software around team-sized boundaries.
- ☐ Ensure teams can maintain, operate, and improve their systems.
- ☐ Prioritize humane and sustainable architecture choices.

9. Build Supporting Capabilities

- ☐ Provide team coaching.
- ☐ Provide mentoring, especially for senior staff.
- ☐ Improve service management practices.
- ☐ Maintain strong documentation.
- ☐ Invest in process improvement.
- ☐ Support continuous delivery and operability.
- ☐ Encourage pairing, mobbing, testing, and review practices.
- ☐ Avoid blame-based incident reviews.

10. Create the Right Organizational Conditions

- ☐ Build a healthy organizational culture.
- ☐ Make it safe for people to speak up about problems.
- ☐ Support professional development for individuals and teams.
- ☐ Avoid harmful CapEx/OpEx funding splits.
- ☐ Reduce project-driven deadlines and large-batch budgeting where possible.
- ☐ Allocate training budgets to teams, not only individuals.
- ☐ Provide clear business vision and priorities.
- ☐ Explain why priorities were chosen.

11. Practice and Evolve

- ☐ Teach teams the principles behind the new ways of working.
- ☐ Practice different interaction modes deliberately.
- ☐ Explain why some teams work closely together while others stay apart.
- ☐ Review team boundaries as technology and product needs change.
- ☐ Expect collaboration during discovery periods.
- ☐ Push stable patterns into platforms and supporting tooling.
- ☐ Treat team topology as dynamic, not fixed.
- ☐ Continuously sense whether the organization needs to adapt.

12. Final Readiness Check

- ☐ Teams are aligned to business streams.
- ☐ Team types are limited and intentional.
- ☐ Interaction modes are explicit.
- ☐ Cognitive load is manageable.
- ☐ Architecture supports team ownership.
- ☐ Platform support is useful but not excessive.
- ☐ Engineering practices support fast flow.
- ☐ Culture, funding, and leadership support the model.
- ☐ Teams understand how and why the organization is changing.

